WEEK 1 LAB

First LLM API Call

1. Project Overview

The objective of Week 1 was to make a first connection between a Python program and a Large Language Model (LLM) through an API. The program loads a Gemini API key from a local .env file, creates a Google Gen AI client, sends a prompt asking for a simple explanation of Artificial Intelligence, and prints the generated response in the terminal.

2. Tools and Technologies

Tool / Library	Purpose
Python	Programming language used to write the application
Visual Studio Code	Code editor and integrated terminal
Google Gemini API	Cloud-based LLM service
google-genai	Python SDK used to communicate with Gemini
python-dotenv	Loads environment variables from the .env file
Virtual environment (.venv)	Keeps project dependencies isolated

3. Project Folder Structure

The project contains the following main files and folders:

```
WEEK1-LLM-CALL/
├── .venv/                  # Python virtual environment
├── .env                    # Gemini API key (private)
├── .gitignore              # Files excluded from Git
├── main.py                 # Python program / entry point
└── requirements file       # Project dependencies
```

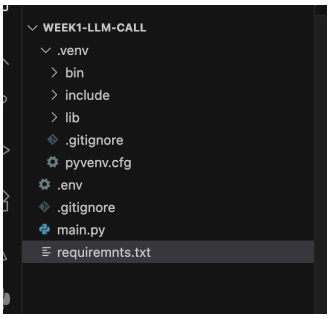


Figure 1. WEEK1-LLM-CALL project structure in Visual Studio Code.

4. Environment Setup

1. Open the WEEK1-LLM-CALL folder in Visual Studio Code.
2. Open the integrated terminal using Terminal → New Terminal.
3. Activate the existing virtual environment from the project folder:

```
source .venv/bin/activate
```

The terminal prompt should display (.venv) when the environment is active.

Install the packages required by the Gemini implementation (if they are not already installed):

```
python -m pip install google-genai python-dotenv
```

The project was run with the existing .venv; a second virtual environment was not needed.

5. Configuration: .env

The Gemini API key is read from a .env file in the project folder. The key is not written directly in main.py. The file should contain the following line, with the placeholder replaced by a newly generated key:

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

6. .gitignore Configuration

The .gitignore file prevents private credentials and generated environment files from being committed:

```
.env
.venv/
__pycache__/
.pytest_cache/
```

7. Dependencies: requirements.txt

```
google-genai
python-dotenv
```

Install the listed dependencies from the activated environment with:

```
python -m pip install -r requirements.txt
```

8. Source Code: main.py

The following is the main.py code loads the API key, initializes the Gemini client, sends the prompt to the gemini-3.5-flash-lite model, and prints the returned text.

```
import os
from dotenv import load_dotenv
from google import genai

# Load variables from the .env file
load_dotenv()

# Read the Gemini API key
api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    raise ValueError("GEMINI_API_KEY was not found. Check your .env file.")

# Initialize the Gemini client
client = genai.Client(api_key=api_key)

# Send a prompt to the model
response = client.models.generate_content(
    model="gemini-3.5-flash-lite",
    contents="What is Artificial Intelligence? Explain in simple words."
)

# Display the generated response
print("\nLLM Response:")
print(response.text)
```

9. Understanding the Program

Code element	Purpose
import os	Provides access to environment variables.
from dotenv import load_dotenv	Imports the function used to load values from the .env file.
from google import genai	Imports the Google Gen AI Python SDK.
load_dotenv()	Loads variables defined in the local .env file.
os.getenv("GEMINI_API_KEY")	Reads the Gemini API key from the environment.
genai.Client(api_key=api_key)	Creates an authenticated Gemini client.
client.models.generate_content(...)	Sends the prompt to the selected Gemini model.
response.text	Retrieves the generated text from the model response.
print(...)	Displays the response in the terminal.

10. Running the Application

From the VS Code integrated terminal, inside the WEEK1-LLM-CALL folder, activate the environment and run:

```
source .venv/bin/activate
python main.py
```

The program prints the heading LLM Response and then displays the text generated by Gemini in response to the prompt: “What is Artificial Intelligence? Explain in simple words.”

11. Execution and Output

The screenshot below shows the virtual environment activated in the terminal, the command python main.py, and the generated LLM response. The output explains AI in simple terms and gives everyday examples such as recommendation systems, voice assistants, video games and spam filters.

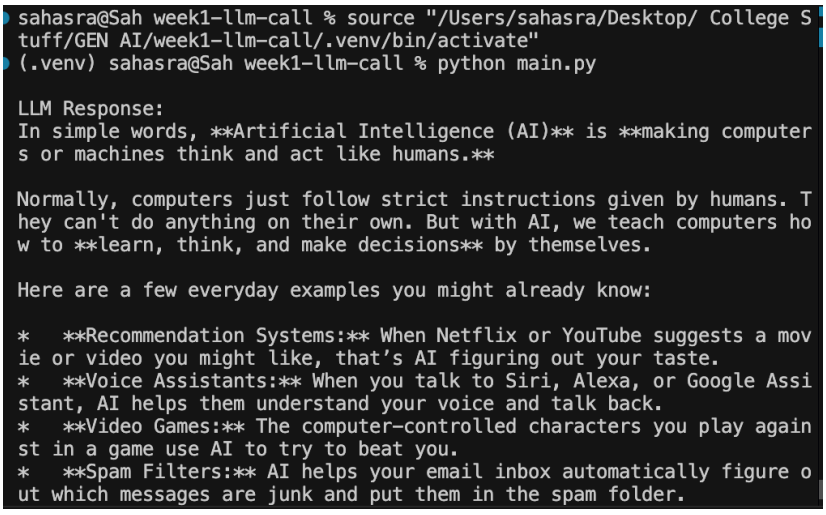


Figure 2. Successful execution of main.py and Gemini-generated response in the VS Code terminal.

12. Result

The Python application successfully connected to the Gemini API using the Google Gen AI SDK. The API key was loaded from the .env file, the prompt was sent to the gemini-3.5-flash-lite model, and the generated explanation of Artificial Intelligence was displayed in the terminal.

13. Conclusion

This lab demonstrated the basic workflow of connecting a Python program to a cloud-based Large Language Model. It introduced API authentication, environment-variable configuration, Python package installation, sending prompts to a model, and displaying generated responses. The successful terminal output confirms that the first LLM API call was completed.